



Presentation on Project-Based Learning

SEE

Data Communications and Networking

(ET363IA)

**Title: Implement TCP Tahoe congestion control with Kruskal's Algorithm and
Stop-
and-Wait protocol for optimized flow control.**

Sl. No	Name of the Student	USN
1.	Naanya Sharma	1RV22ET028
2.	Palash Tamrakar	1RV22ET030



**RV College of
Engineering®**

Go, change the world

Table of Contents

1. Introduction
2. Literature Survey
3. Problem Statement
4. Objectives
5. Theory and Design
6. Methodology
7. Implementation and Results
8. References



Introduction

In modern networking, understanding how data flows efficiently and reliably across complex systems is crucial. This project presents an interactive Python-based simulator that integrates three key networking concepts: TCP Tahoe for congestion control, Stop-and-Wait ARQ for reliable data delivery, and Kruskal's algorithm for optimal routing using a Minimum Spanning Tree (MST). Through a graphical user interface (GUI), users can input custom data, visualize packet behavior, monitor real-time logs, and explore protocol dynamics. The tool serves as both an educational platform and a practical demonstration of how these protocols collaborate to ensure optimized flow control in digital communication networks.



Literature Survey

Paper Title	Authors	Publication Details	Summary
<i>TCP Congestion Control Algorithm for Low Latency and High Throughput</i>	H. Kim, J. Kim, S. Choi	IEEE Access, Vol. 8, 2020, pp. 104234–104246	Discusses improvements over TCP Tahoe for high-speed networks. Serves as a benchmark to evaluate newer, adaptive congestion control mechanisms.
<i>An Enhanced ARQ Scheme for Ultra-Reliable Low-Latency Communications</i>	J. Li, Y. Jiang	IEEE Transactions on Communications, Vol. 69, No. 2, Feb 2021	Proposes a modified ARQ method suitable for low-latency use cases. Highlights the relevance of Stop-and-Wait as a foundational reliability protocol.
<i>Efficient Spanning Tree Construction for Wireless Mesh Networks</i>	S. Raj, A. Joshi	Wireless Networks, Vol. 28, No. 2, Feb. 2022, pp. 1231–1245	Explores MST construction methods like Kruskal’s for optimized routing. Demonstrates their utility in wireless and resource-constrained networks.
<i>Visual Simulators for Learning Networking Protocols: A Comparative Study</i>	M. Patel, R. Shah	Education and Information Technologies, Vol. 27, Nov. 2022	Analyzes educational tools for protocol simulation. Validates the use of visual platforms for learning TCP, ARQ, and routing algorithms effectively.



**RV College of
Engineering®**

Go, change the world

Problem Statement

Textbook-based learning often falls short in visualizing dynamic networking protocols like TCP Tahoe, Stop-and-Wait ARQ, and Kruskal's MST. Key behaviors such as congestion control, retransmissions, and optimal routing are hard to grasp without simulation. This project aims to bridge that gap by creating an interactive, GUI-based simulator that helps users visualize and understand these protocols in real time.



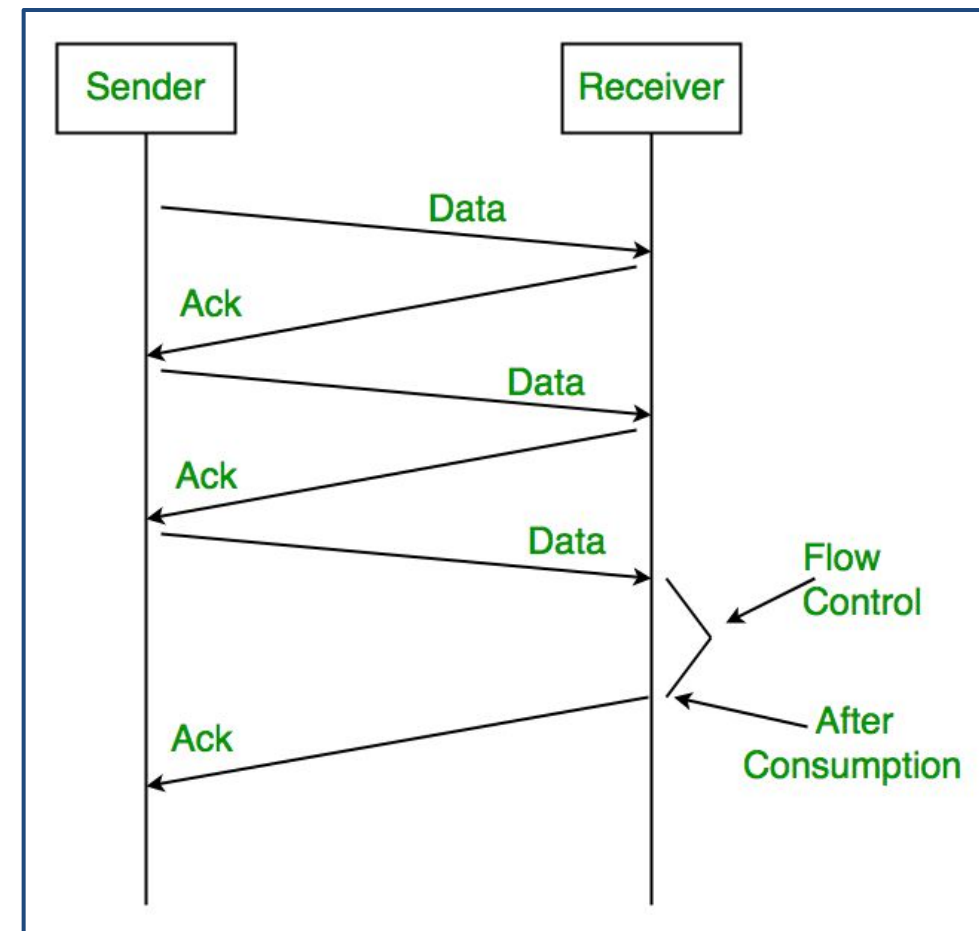
Objectives

- To simulate **TCP Tahoe** and visualize congestion control through CWND behavior.
- To implement **Stop-and-Wait Protocol** for reliable data transmission with timeouts and ACKs.
- To compute and animate **Kruskal's Minimum Spanning Tree** for optimal routing.
- To develop an interactive **GUI** for real-time user input, visualization, and control.



Theory and Design

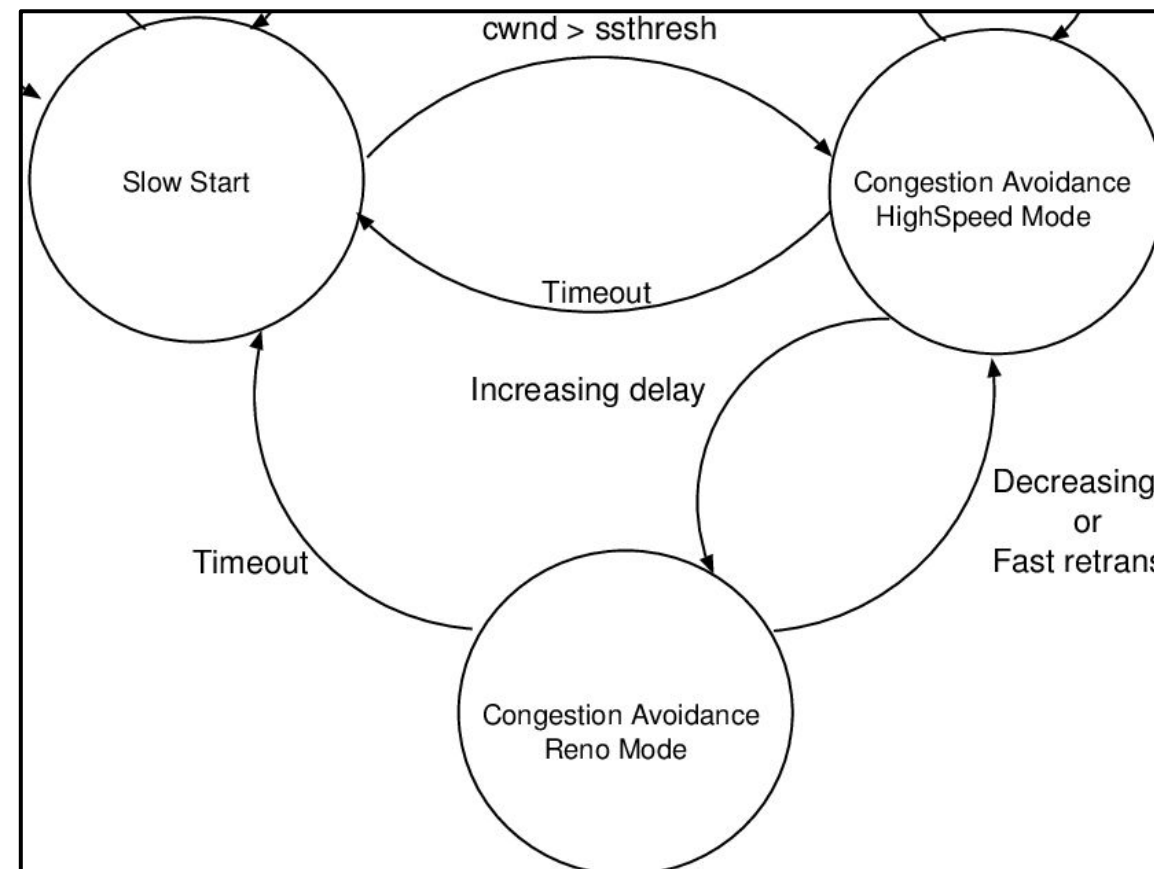
Stop-and-Wait Protocol is a basic reliability protocol where each packet is sent individually, and the sender waits for an ACK before sending the next. If the ACK is not received in time, the packet is retransmitted. In this project, the protocol is simulated with a loop that handles timeouts and retries (up to three attempts). The flow is visualized through animations, and real-time logs display each transmission and acknowledgment, demonstrating the concept of reliable communication.





Theory and Design

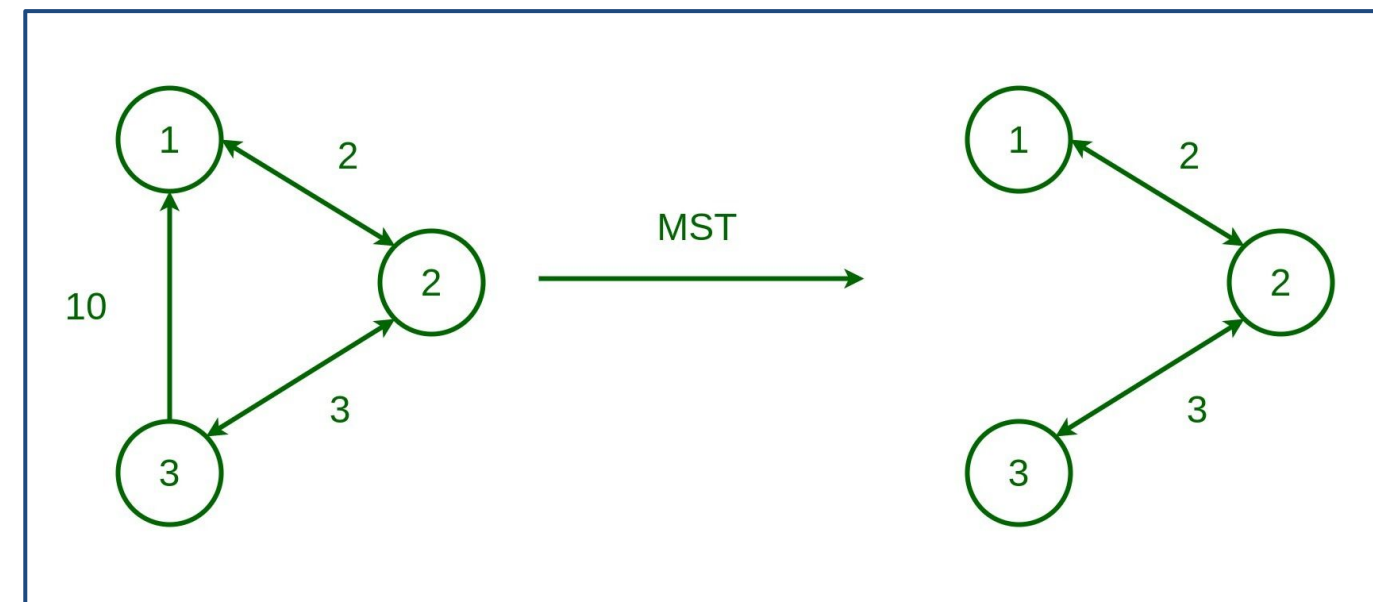
TCP Tahoe manages congestion through slow start, congestion avoidance, and fast retransmit. It increases the congestion window (CWND) gradually to probe available bandwidth and reduces it sharply upon packet loss. In the simulator, these phases are implemented with adjustable MSS and MTU values. Packet transmission is animated, and CWND behavior is logged in real time, allowing users to observe how the algorithm responds to network conditions.





Theory and Design

Kruskal's algorithm finds the Minimum Spanning Tree by selecting the lowest-weight edges that don't form cycles. This results in an efficient routing path with minimal total cost. The simulator uses NetworkX to model the graph and apply Kruskal's logic. The MST is dynamically highlighted using Matplotlib, and users can clearly see how optimal paths are selected in a network, reinforcing graph theory concepts in routing.





Methodology

- **User Input & Encoding:**

The user inputs a word and selects the encoding format (ASCII/UTF-8/4-bit). The word is then split into 4 binary segments, each appended with a 20-bit header to form datagrams.

- **Network Setup & MST Calculation:**

A network topology is either selected or created by the user. The graph is visualized using NetworkX, and Kruskal's algorithm is applied to compute the Minimum Spanning Tree (MST), which is highlighted for optimal routing.

- **TCP Tahoe Simulation:**

The simulator models the behavior of TCP Tahoe, including slow start, congestion avoidance, and fast retransmit. Packet transmission is animated, and CWND adjustments are displayed dynamically. Parameters like MSS, MTU, and loss rate can be modified in real time.



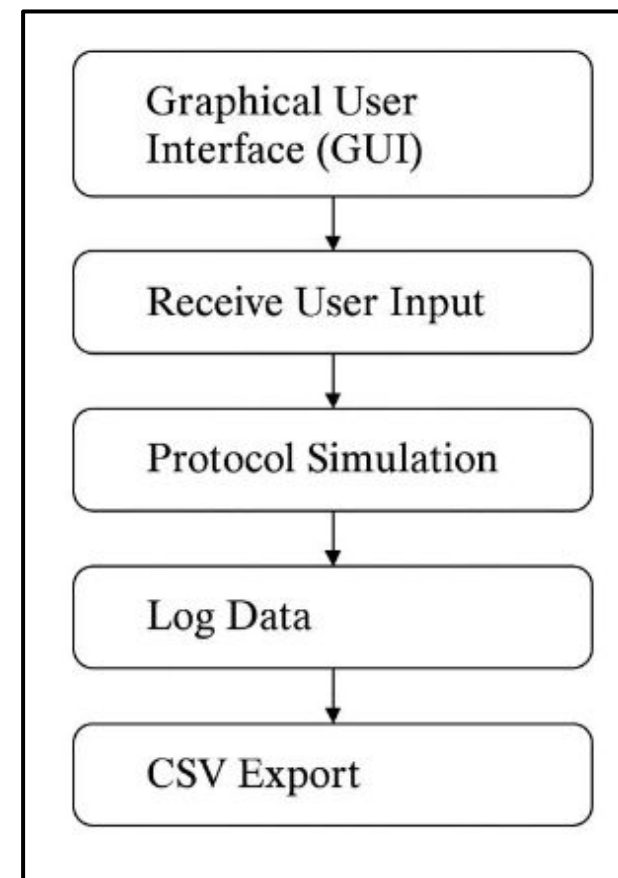
Methodology

- **Stop-and-Wait ARQ Simulation:**

Each packet is sent individually, and the system waits for an ACK before proceeding. If a timeout occurs, retransmissions are triggered (up to 3 times). The process is animated, showing packet and ACK movement clearly.

- **Logging & Output Visualization:**

Logs of transmission events, ACKs, CWND states, and retransmissions are displayed live in the GUI. These logs can also be exported as CSV files for analysis. Graphs and packet statistics offer further insights into protocol behavior.





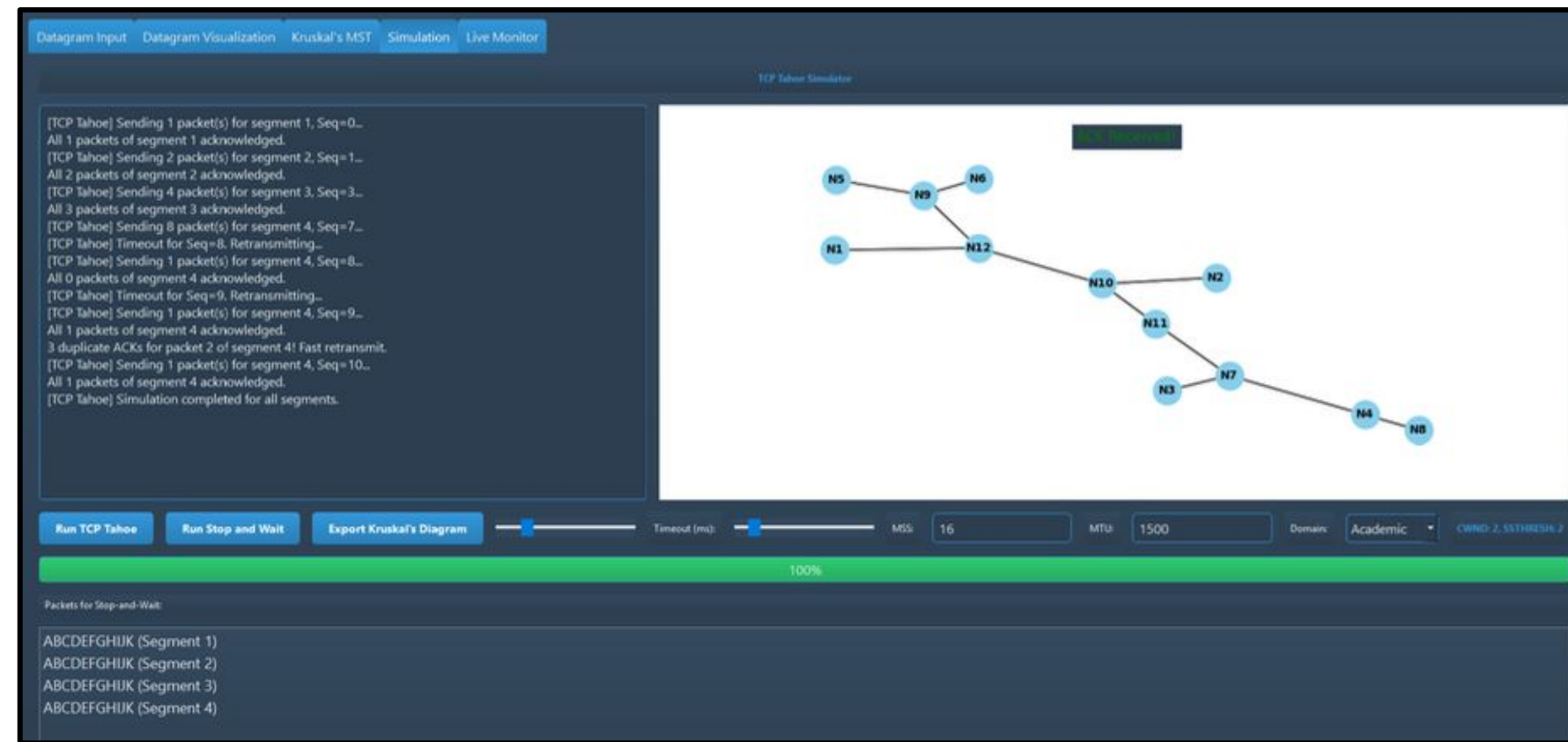
Implementation And Results

The simulator was implemented in Python using the PyQt5 framework for the GUI and NetworkX for graph creation and routing. The core components—**TCP Tahoe**, **Stop-and-Wait ARQ**, and **Kruskal's MST**—were developed as modular classes.

- Users input a word and choose the encoding format; the system then forms binary datagrams with headers.
- The network graph is visualized and processed through Kruskal's algorithm to generate an MST.
- TCP Tahoe simulation animates packet transmission, CWND growth, packet loss, and recovery.
- Stop-and-Wait ARQ handles individual packet delivery with ACKs, timeouts, and retransmissions.
- The GUI displays live logs, animations, and allows CSV export of TCP transmission data.



Implementation And Results



This shows the TCP Tahoe simulation panel in action. The user sends a segmented word through the network, and the simulation logs the behavior of the TCP congestion control mechanism. CWND increases gradually during the slow start phase, and the logs show how packets are acknowledged. A fast retransmit is triggered for segment 4, demonstrating how TCP Tahoe handles packet loss. The path on the graph is animated with real-time acknowledgments, and key metrics like MSS and MTU are adjustable.



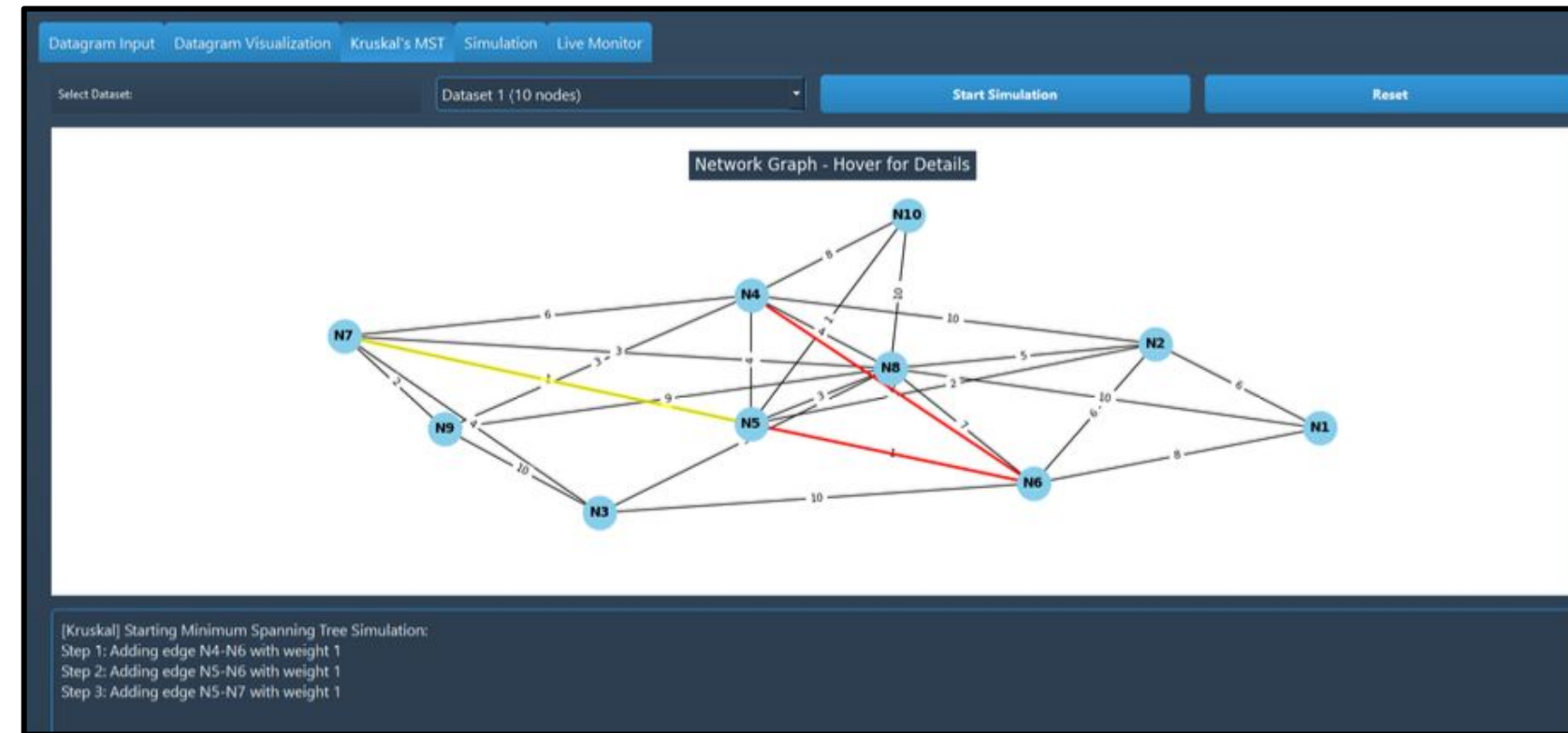
Implementation And Results

	Dataset	Nodes	
1	Dataset 1 (10 nodes)	N1, N2, N3, N4, N5, N6, N7, N8, N9, N10	N4-N6-1, N5-N6-1, N5-N7-1, N5-N10-1, N2-N5-2, N7-N9-2, N3-N8-3, N4-N9-3, N5-N8-3, N7-N8-3,
2	Dataset 2 (13 nodes)	N1, N2, N3, N4, N5, N6, N7, N8, N9, N10, N11, N12, N13	N1-N2-10, N1-N3-6, N1-N4-7, N1-N6-10, N1-N7-3, N1-N8-9, N1-N10-6, N1-N13-6, N2-N4-4, N2-N5-

This illustrates the input stage where users define the simulation environment. The user can choose between preset network topologies or manually enter a custom graph. Additional fields allow input of the message to be segmented, selection of encoding type (e.g., ASCII 8-bit), number of segments, and the simulated noise level in the channel. This configuration ensures the system is flexible and allows controlled experimentation across various network scenarios.



Implementation And Results



In this simulation stage, Kruskal's algorithm is applied to the selected graph dataset. The MST is calculated step by step, with selected edges (e.g., N4-N6, N5-N6, N5-N7) being highlighted in red and yellow. The edge weights are displayed, and the real-time console at the bottom logs the construction of the tree. This visual demonstration allows users to clearly understand how the MST is formed and how optimal paths are chosen for efficient routing.



References

1. Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson Education.
2. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson Education.
3. H. Kim, J. Kim, and S. Choi, “TCP Congestion Control Algorithm for Low Latency and High Throughput,” *IEEE Access*, vol. 8, pp. 104234–104246, 2020.
4. J. Li and Y. Jiang, “An Enhanced ARQ Scheme for Ultra-Reliable Low-Latency Communications,” *IEEE Transactions on Communications*, vol. 69, no. 2, pp. 1343–1355, Feb. 2021.
5. S. Raj and A. Joshi, “Efficient Spanning Tree Construction for Wireless Mesh Networks,” *Wireless Networks*, vol. 28, no. 2, pp. 1231–1245, 2022.
6. M. Patel and R. Shah, “Visual Simulators for Learning Networking Protocols: A Comparative Study,” *Education and Information Technologies*, vol. 27, pp. 10567–10585, Nov. 2022.